

Judge TCG v3.0 — Cursor Context File

Generated: 2026-06-06

Repository: S:-judge

Status: Fases 1-5 implementadas, Fase 6 em planejamento

Total Tests: 24 pytest + 109 vitest

1. VISÃO GERAL DO PROJETO

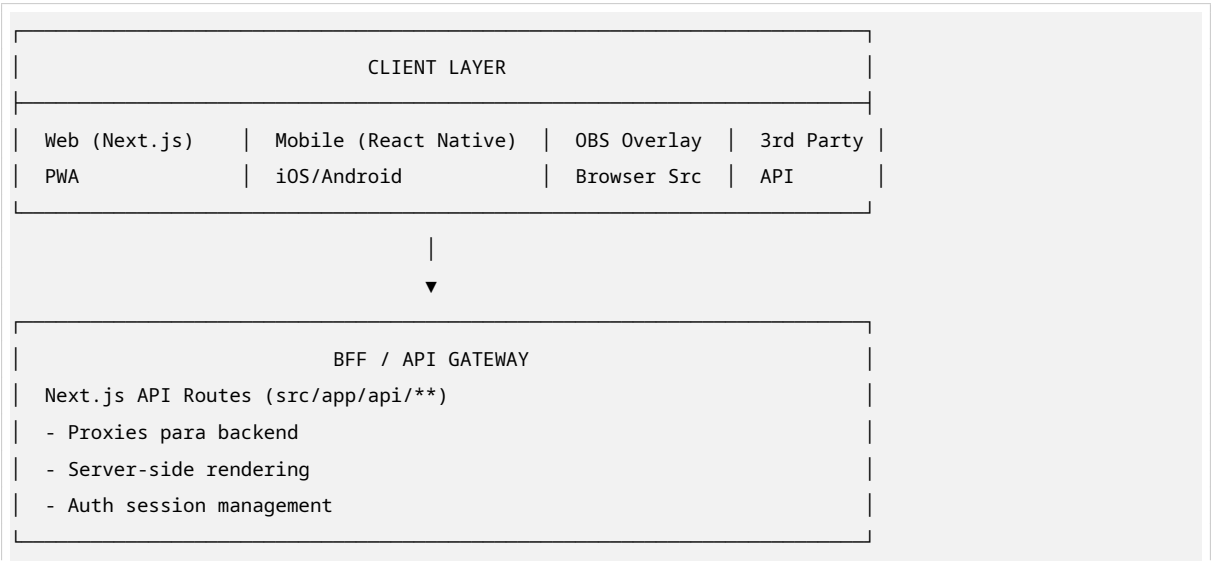
Judge TCG é uma plataforma completa de gerenciamento de torneios de Trading Card Games (TCG), suportando Pokémon TCG, Disney Lorcana, Magic: The Gathering (MTG) e Star Wars Unlimited (SWU).

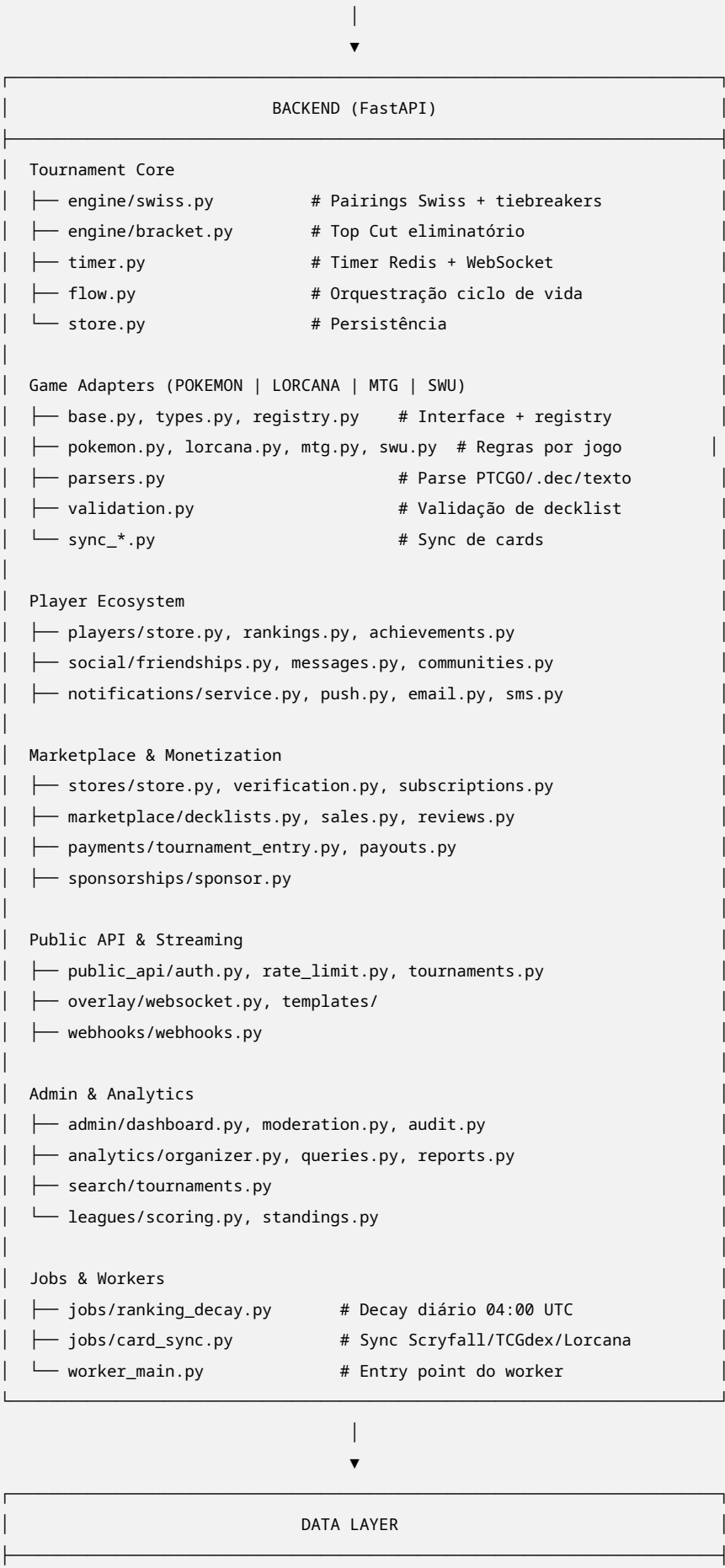
A plataforma evoluiu de uma ferramenta de execução de torneios para um ecossistema completo com marketplace, streaming overlay, API pública, monetização, engajamento social e PWA.

Stack Tecnológica

- **Backend:** Python (FastAPI), PostgreSQL (Supabase), Redis, WebSocket/SSE
- **Frontend:** Next.js 14 (App Router), React, TypeScript, Tailwind CSS
- **Mobile:** React Native (Fase 6 — planejado)
- **Infra:** Docker, Supabase (DB + Auth + Realtime), Vercel/Cloud
- **Pagamentos:** Stripe (cartão, PIX, PayPal)
- **Notificações:** Firebase Cloud Messaging (Push), SendGrid (Email), Twilio (SMS)
- **AI:** Scikit-learn / AWS SageMaker (Fase 6 — planejado)

2. ARQUITETURA





PostgreSQL (Supabase)
├─ Auth/RLS (Row Level Security) em todas as tabelas
├─ games, game_formats, card_catalog
├─ tournaments, tournament_participants, tournament_rounds
├─ pairings, brackets, bracket_matches
├─ player_profiles, player_game_stats, rankings, achievements
├─ stores, store_subscriptions, store_followers
├─ marketplace_decklists, decklist_sales, reviews
├─ payments, payouts
├─ api_keys, webhooks, sponsorships
├─ notifications, notification_preferences
├─ friendships, messages, communities, community_members
├─ leagues, league_events, league_standings
└─ infraction_reports, judge_calls, judge_certifications
Redis
├─ Timer state (round timers)
├─ Cache de leaderboards
├─ Rate limiting (API pública)
└─ Pub/Sub (WebSocket broadcast)

3. DATABASE SCHEMA COMPLETO

3.1 Core Tables

```
-- Jogos suportados
CREATE TABLE games (
  code VARCHAR(10) PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  active BOOLEAN DEFAULT true,
  card_sync_endpoint TEXT,
  card_sync_frequency INTERVAL,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Seed: POKEMON, LORCANA, MTG, SWU

-- Formatos por jogo (13 formatos seed)
CREATE TABLE game_formats (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  game_code VARCHAR(10) REFERENCES games(code),
  code VARCHAR(20) NOT NULL,
  name VARCHAR(50) NOT NULL,
  decklist_required BOOLEAN DEFAULT true,
  decklist_validation_rules JSONB,
  default_timer_minutes INT,
  default_match_type VARCHAR(5), -- B01, B03, B05
  default_swiss_rounds VARCHAR(10),
  default_top_cut INT,
  UNIQUE(game_code, code)
```

```

);

-- Catálogo de cards unificado
CREATE TABLE card_catalog (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  game_code VARCHAR(10) REFERENCES games(code),
  external_id VARCHAR(50) NOT NULL,
  set_code VARCHAR(10),
  set_name VARCHAR(100),
  card_number VARCHAR(10),
  name VARCHAR(200) NOT NULL,
  normalized_name VARCHAR(200),
  rarity VARCHAR(20),
  card_type VARCHAR(50),
  game_specific_type VARCHAR(50),
  legality JSONB,
  image_url TEXT,
  game_data JSONB,
  last_synced_at TIMESTAMP,
  UNIQUE(game_code, external_id)
);

CREATE INDEX idx_cards_normalized_name ON card_catalog(normalized_name);
CREATE INDEX idx_cards_game_legality ON card_catalog(game_code, (legality->>'STANDARD'));

```

3.2 Tournament Tables

```

-- Torneios
CREATE TABLE tournaments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organizer_id UUID NOT NULL,
  store_id UUID, -- FK para stores (Fase 5)
  name VARCHAR(200) NOT NULL,
  game_code VARCHAR(10) REFERENCES games(code),
  format_code VARCHAR(20) NOT NULL,
  status VARCHAR(20) DEFAULT 'draft', -- draft | published | check_in | in_progress | between_rounds
  ↳ | swiss_complete | bracket_active | finalized | canceled
  config JSONB,
  game_code VARCHAR(10),
  format_code VARCHAR(20),
  match_type VARCHAR(5) DEFAULT 'B03',
  timer_minutes INT DEFAULT 55,
  top_cut INT,
  swiss_rounds VARCHAR(10) DEFAULT 'auto',
  max_participants INT DEFAULT 0,
  registration_fee_cents INT DEFAULT 0,
  currency VARCHAR(3) DEFAULT 'BRL',
  prize_description TEXT,
  custom_rules TEXT,
  logo_id UUID,
  location JSONB,
  is_official BOOLEAN DEFAULT false,

```

```

    is_private BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Participantes
CREATE TABLE tournament_participants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tournament_id UUID REFERENCES tournaments(id) ON DELETE CASCADE,
    player_id UUID, -- NULL = inscrição rápida (guest)
    guest_info JSONB,
    decklist_id UUID,
    status VARCHAR(20) DEFAULT 'registered', -- registered | checked-in | dropped | disqualified
    payment_status VARCHAR(20) DEFAULT 'pending', -- pending | paid | refunded
    wins INT DEFAULT 0,
    losses INT DEFAULT 0,
    draws INT DEFAULT 0,
    match_points INT DEFAULT 0,
    omw_percent DECIMAL(5,2) DEFAULT 0,
    gw_percent DECIMAL(5,2) DEFAULT 0,
    ogw_percent DECIMAL(5,2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Rodadas
CREATE TABLE tournament_rounds (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tournament_id UUID REFERENCES tournaments(id) ON DELETE CASCADE,
    round_number INT NOT NULL,
    status VARCHAR(20) DEFAULT 'pending', -- pending | active | completed
    started_at TIMESTAMP,
    ended_at TIMESTAMP,
    timer_duration_seconds INT DEFAULT 3300,
    timer_extensions_seconds INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(tournament_id, round_number)
);

-- Pairings
CREATE TABLE pairings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    round_id UUID REFERENCES tournament_rounds(id) ON DELETE CASCADE,
    table_number INT NOT NULL,
    player1_id UUID REFERENCES tournament_participants(id),
    player2_id UUID REFERENCES tournament_participants(id), -- NULL = bye
    player1_wins INT DEFAULT 0,
    player2_wins INT DEFAULT 0,
    draws INT DEFAULT 0,
    status VARCHAR(20) DEFAULT 'pending', -- pending | reported | confirmed | disputed
    reported_by UUID REFERENCES tournament_participants(id),
    confirmed_by UUID REFERENCES tournament_participants(id),

```

```

    is_bye BOOLEAN DEFAULT false,
    is_forced_rematch BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Brackets (Top Cut)
CREATE TABLE brackets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tournament_id UUID REFERENCES tournaments(id) ON DELETE CASCADE,
    format VARCHAR(20) DEFAULT 'SINGLE_ELIMINATION',
    status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE bracket_matches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    bracket_id UUID REFERENCES brackets(id) ON DELETE CASCADE,
    round_number INT NOT NULL,
    match_number INT NOT NULL,
    player1_id UUID REFERENCES tournament_participants(id),
    player2_id UUID REFERENCES tournament_participants(id),
    winner_id UUID REFERENCES tournament_participants(id),
    next_match_id UUID REFERENCES bracket_matches(id),
    table_number INT,
    status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT NOW()
);

```

3.3 Player Ecosystem Tables

```

-- Perfis de jogador
CREATE TABLE player_profiles (
    id UUID PRIMARY KEY REFERENCES auth.users(id),
    handle VARCHAR(30) UNIQUE NOT NULL,
    display_name VARCHAR(50) NOT NULL,
    avatar_url TEXT,
    bio TEXT,
    favorite_game VARCHAR(10),
    city VARCHAR(50),
    country VARCHAR(2),
    timezone VARCHAR(50) DEFAULT 'America/Sao_Paulo',
    locale VARCHAR(10) DEFAULT 'pt-BR',
    privacy_level VARCHAR(20) DEFAULT 'public', -- public | friends | private
    role VARCHAR(20) DEFAULT 'player', -- player | organizer | judge | admin
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Estatísticas por jogo
CREATE TABLE player_game_stats (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

player_id UUID REFERENCES player_profiles(id),
game_code VARCHAR(10) REFERENCES games(code),
tournaments_played INT DEFAULT 0,
tournaments_won INT DEFAULT 0,
top_cuts INT DEFAULT 0,
matches_won INT DEFAULT 0,
matches_lost INT DEFAULT 0,
matches_drawn INT DEFAULT 0,
game_wins INT DEFAULT 0,
game_losses INT DEFAULT 0,
best_finish_placement INT,
best_finish_tournament_id UUID,
best_finish_date TIMESTAMP,
favorite_format VARCHAR(20),
total_prizes VARCHAR(50),
UNIQUE(player_id, game_code)
);

-- Conquistas
CREATE TABLE achievements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(50) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    icon VARCHAR(50),
    rarity VARCHAR(20) -- common | rare | epic | legendary
);

CREATE TABLE player_achievements (
    player_id UUID REFERENCES player_profiles(id),
    achievement_id UUID REFERENCES achievements(id),
    unlocked_at TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (player_id, achievement_id)
);

-- Rankings
CREATE TABLE rankings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    player_id UUID REFERENCES player_profiles(id),
    game_code VARCHAR(10) REFERENCES games(code),
    format VARCHAR(20) NOT NULL,
    points INT DEFAULT 0,
    tier VARCHAR(20) DEFAULT 'Bronze',
    division INT DEFAULT 1,
    matches_played INT DEFAULT 0,
    last_updated TIMESTAMP DEFAULT NOW(),
    UNIQUE(player_id, game_code, format)
);

-- Histórico de torneios
CREATE TABLE tournament_results (

```

```

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    player_id UUID REFERENCES player_profiles(id),
    tournament_id UUID REFERENCES tournaments(id),
    placement INT,
    match_wins INT DEFAULT 0,
    match_losses INT DEFAULT 0,
    match_draws INT DEFAULT 0,
    points_earned INT DEFAULT 0,
    decklist_id UUID,
    prize VARCHAR(100),
    created_at TIMESTAMP DEFAULT NOW()
);

```

3.4 Social Tables

```

-- Amigos
CREATE TABLE friendships (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    requester_id UUID REFERENCES player_profiles(id),
    addressee_id UUID REFERENCES player_profiles(id),
    status VARCHAR(20) DEFAULT 'pending', -- pending | accepted | blocked
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(requester_id, addressee_id)
);

-- Mensagens
CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sender_id UUID REFERENCES player_profiles(id),
    receiver_id UUID REFERENCES player_profiles(id),
    content TEXT NOT NULL,
    read BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_messages_conversation ON messages(sender_id, receiver_id, created_at);

-- Comunidades
CREATE TABLE communities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    description TEXT,
    game_code VARCHAR(10) REFERENCES games(code),
    created_by UUID REFERENCES player_profiles(id),
    avatar_url TEXT,
    member_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE community_members (
    community_id UUID REFERENCES communities(id),

```



```

    player_id UUID REFERENCES player_profiles(id),
    role VARCHAR(20) DEFAULT 'member', -- member | moderator | admin
    joined_at TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (community_id, player_id)
);

```

3.5 Marketplace & Monetization Tables

```

-- Lojas
CREATE TABLE stores (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id UUID REFERENCES player_profiles(id),
    name VARCHAR(100) NOT NULL,
    slug VARCHAR(50) UNIQUE NOT NULL,
    description TEXT,
    logo_url TEXT,
    banner_url TEXT,
    address TEXT,
    city VARCHAR(50),
    state VARCHAR(50),
    country VARCHAR(2),
    lat DECIMAL(10, 8),
    lng DECIMAL(11, 8),
    phone VARCHAR(20),
    email VARCHAR(100),
    website TEXT,
    discord TEXT,
    verification_status VARCHAR(20) DEFAULT 'pending', -- pending | verified | rejected
    verified_at TIMESTAMP,
    verified_by UUID REFERENCES player_profiles(id),
    subscription_plan VARCHAR(20) DEFAULT 'free', -- free | pro | enterprise
    subscription_expires_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE store_subscriptions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    store_id UUID REFERENCES stores(id),
    plan VARCHAR(20) NOT NULL,
    amount_cents INT NOT NULL,
    currency VARCHAR(3) DEFAULT 'BRL',
    status VARCHAR(20) DEFAULT 'active',
    started_at TIMESTAMP DEFAULT NOW(),
    expires_at TIMESTAMP,
    canceled_at TIMESTAMP
);

CREATE TABLE store_followers (
    store_id UUID REFERENCES stores(id),
    player_id UUID REFERENCES player_profiles(id),
    followed_at TIMESTAMP DEFAULT NOW(),

```

```

    PRIMARY KEY (store_id, player_id)
);

-- Marketplace de decklists
CREATE TABLE marketplace_decklists (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    seller_id UUID REFERENCES player_profiles(id),
    tournament_id UUID REFERENCES tournaments(id),
    game_code VARCHAR(10) REFERENCES games(code),
    format VARCHAR(20) NOT NULL,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    decklist_data JSONB NOT NULL,
    price_cents INT NOT NULL,
    currency VARCHAR(3) DEFAULT 'BRL',
    status VARCHAR(20) DEFAULT 'active', -- active | sold | withdrawn
    sales_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE decklist_sales (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    decklist_id UUID REFERENCES marketplace_decklists(id),
    buyer_id UUID REFERENCES player_profiles(id),
    price_cents INT NOT NULL,
    platform_fee_cents INT NOT NULL,
    seller_receives_cents INT NOT NULL,
    status VARCHAR(20) DEFAULT 'completed',
    created_at TIMESTAMP DEFAULT NOW()
);

-- Reviews
CREATE TABLE reviews (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reviewer_id UUID REFERENCES player_profiles(id),
    target_type VARCHAR(20) NOT NULL, -- tournament | store | organizer | decklist
    target_id UUID NOT NULL,
    rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
    title VARCHAR(200),
    comment TEXT,
    categories JSONB,
    helpful_count INT DEFAULT 0,
    reported BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Pagamentos
CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tournament_id UUID REFERENCES tournaments(id),
    player_id UUID REFERENCES player_profiles(id),

```

```

    amount_cents INT NOT NULL,
    currency VARCHAR(3) DEFAULT 'BRL',
    platform_fee_cents INT,
    organizer_receives_cents INT,
    gateway VARCHAR(20), -- stripe | paypal | pix
    gateway_payment_id VARCHAR(100),
    status VARCHAR(20) DEFAULT 'pending', -- pending | paid | refunded | failed
    paid_at TIMESTAMP,
    refunded_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE payouts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organizer_id UUID REFERENCES player_profiles(id),
    amount_cents INT NOT NULL,
    currency VARCHAR(3),
    status VARCHAR(20) DEFAULT 'pending', -- pending | processing | completed
    method VARCHAR(20),
    processed_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Patrocínios
CREATE TABLE sponsorships (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sponsor_id UUID REFERENCES player_profiles(id),
    tournament_id UUID REFERENCES tournaments(id),
    amount_cents INT NOT NULL,
    currency VARCHAR(3) DEFAULT 'BRL',
    status VARCHAR(20) DEFAULT 'pending',
    benefits JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

```

3.6 Public API & Infrastructure Tables

```

-- API Keys (Developer Portal)
CREATE TABLE api_keys (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id UUID REFERENCES player_profiles(id),
    key_hash VARCHAR(100) NOT NULL,
    key_prefix VARCHAR(10) NOT NULL,
    plan VARCHAR(20) DEFAULT 'free', -- free | pro | enterprise
    rate_limit INT DEFAULT 10000,
    usage_count INT DEFAULT 0,
    last_used_at TIMESTAMP,
    revoked_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

```

```

-- Webhooks
CREATE TABLE webhooks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id UUID REFERENCES player_profiles(id),
  url TEXT NOT NULL,
  events VARCHAR(50)[],
  secret VARCHAR(100),
  active BOOLEAN DEFAULT true,
  last_delivery_at TIMESTAMP,
  last_error TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Notificações
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player_id UUID REFERENCES player_profiles(id),
  type VARCHAR(50) NOT NULL,
  title VARCHAR(200),
  body TEXT,
  data JSONB,
  read BOOLEAN DEFAULT false,
  channels VARCHAR(20)[],
  sent_at TIMESTAMP DEFAULT NOW(),
  read_at TIMESTAMP
);

CREATE TABLE notification_preferences (
  player_id UUID PRIMARY KEY REFERENCES player_profiles(id),
  push_enabled BOOLEAN DEFAULT true,
  email_enabled BOOLEAN DEFAULT true,
  sms_enabled BOOLEAN DEFAULT false,
  event_settings JSONB DEFAULT '{}'
);

-- Push subscriptions (Web Push)
CREATE TABLE push_subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player_id UUID REFERENCES player_profiles(id),
  endpoint TEXT NOT NULL,
  p256dh VARCHAR(200),
  auth VARCHAR(200),
  created_at TIMESTAMP DEFAULT NOW()
);

-- Ligas
CREATE TABLE leagues (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL,
  game_code VARCHAR(10) REFERENCES games(code),
  format VARCHAR(20) NOT NULL,

```

```

        organizer_id UUID REFERENCES player_profiles(id),
        season_start DATE,
        season_end DATE,
        scoring_rules JSONB,
        prize_structure JSONB,
        status VARCHAR(20) DEFAULT 'active',
        created_at TIMESTAMP DEFAULT NOW()
    );

CREATE TABLE league_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    league_id UUID REFERENCES leagues(id),
    name VARCHAR(100),
    event_date TIMESTAMP,
    tournament_id UUID REFERENCES tournaments(id),
    points_multiplier DECIMAL(3,1) DEFAULT 1.0
);

CREATE TABLE league_standings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    league_id UUID REFERENCES leagues(id),
    player_id UUID REFERENCES player_profiles(id),
    total_points INT DEFAULT 0,
    events_played INT DEFAULT 0,
    best_finish INT,
    consistency_bonus INT DEFAULT 0,
    updated_at TIMESTAMP DEFAULT NOW()
);

```

3.7 Fase 6 Tables (Planejado)

```

-- Mobile devices
CREATE TABLE mobile_devices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    player_id UUID REFERENCES player_profiles(id),
    platform VARCHAR(10) NOT NULL,
    device_token VARCHAR(200),
    app_version VARCHAR(20),
    last_active_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

-- POS integrations
CREATE TABLE pos_integrations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    store_id UUID REFERENCES stores(id),
    provider VARCHAR(20) NOT NULL,
    config JSONB NOT NULL,
    status VARCHAR(20) DEFAULT 'active',
    created_at TIMESTAMP DEFAULT NOW()
);

```

```
CREATE TABLE pos_transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  pos_integration_id UUID REFERENCES pos_integrations(id),
  type VARCHAR(20) NOT NULL,
  amount_cents INT NOT NULL,
  currency VARCHAR(3),
  metadata JSONB,
  pos_transaction_id VARCHAR(100),
  status VARCHAR(20) DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
-- Judge system
```

```
CREATE TABLE judge_calls (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tournament_id UUID REFERENCES tournaments(id),
  round_id UUID REFERENCES tournament_rounds(id),
  table_number INT,
  caller_id UUID REFERENCES player_profiles(id),
  type VARCHAR(50),
  priority VARCHAR(20) DEFAULT 'medium',
  status VARCHAR(20) DEFAULT 'open',
  assigned_judge_id UUID REFERENCES player_profiles(id),
  description TEXT,
  evidence_urls TEXT[],
  ruling TEXT,
  ruling_judge_id UUID REFERENCES player_profiles(id),
  created_at TIMESTAMP DEFAULT NOW(),
  resolved_at TIMESTAMP
);
```

```
CREATE TABLE judge_certifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player_id UUID REFERENCES player_profiles(id),
  game_code VARCHAR(10) REFERENCES games(code),
  level VARCHAR(20) NOT NULL,
  certified_by UUID REFERENCES player_profiles(id),
  certified_at TIMESTAMP,
  expires_at TIMESTAMP,
  status VARCHAR(20) DEFAULT 'active'
);
```

```
-- Infractions
```

```
CREATE TABLE infraction_reports (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reporter_id UUID REFERENCES player_profiles(id),
  reported_id UUID REFERENCES player_profiles(id),
  tournament_id UUID REFERENCES tournaments(id),
  type VARCHAR(50) NOT NULL,
  description TEXT,
```

```

evidence_urls TEXT[],
status VARCHAR(20) DEFAULT 'pending',
handled_by UUID REFERENCES player_profiles(id),
resolution VARCHAR(20),
resolution_notes TEXT,
created_at TIMESTAMP DEFAULT NOW(),
resolved_at TIMESTAMP
);

CREATE TABLE fair_play_scores (
    player_id UUID REFERENCES player_profiles(id),
    game_code VARCHAR(10) REFERENCES games(code),
    score DECIMAL(3,2) DEFAULT 5.00,
    infractions_count INT DEFAULT 0,
    warnings_count INT DEFAULT 0,
    last_infraction_at TIMESTAMP,
    updated_at TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (player_id, game_code)
);

```

4. API ENDPOINTS COMPLETOS

4.1 Tournament Core

POST	/runtime/judge/tournaments	# Criar torneio
GET	/runtime/judge/tournaments	# Listar torneios
GET	/runtime/judge/tournaments/{id}	# Detalhes do torneio
PUT	/runtime/judge/tournaments/{id}	# Atualizar torneio
DELETE	/runtime/judge/tournaments/{id}	# Cancelar torneio
POST	/runtime/judge/tournaments/{id}/register	# Inscrever-se
POST	/runtime/judge/tournaments/{id}/start-check-in	# Abrir check-in
POST	/runtime/judge/tournaments/{id}/start	# Iniciar torneio
POST	/runtime/judge/tournaments/{id}/finalize	# Finalizar torneio
POST	/runtime/judge/tournaments/{id}/rounds	# Gerar próxima rodada
GET	/runtime/judge/tournaments/{id}/rounds/{n}	# Pairings da rodada
POST	/runtime/judge/tournaments/{id}/rounds/{n}/start	# Iniciar timer
POST	/runtime/judge/tournaments/{id}/rounds/{n}/extend	# Estender tempo
POST	/runtime/judge/tournaments/{id}/rounds/{n}/end	# Finalizar rodada
POST	/runtime/judge/tournaments/{id}/pairings/{pid}/report	# Reportar resultado
POST	/runtime/judge/tournaments/{id}/pairings/{pid}/confirm	# Confirmar resultado
GET	/runtime/judge/tournaments/{id}/standings	# Standings
POST	/runtime/judge/tournaments/{id}/advance-top-cut	# Gerar bracket
GET	/runtime/judge/timers/{roundId}	# Timer da rodada
WS	/ws	# WebSocket geral

SSE	/stream	# Server-Sent Events
-----	---------	----------------------

4.2 Game Adapters

GET	/runtime/judge/games	# Lista jogos
GET	/runtime/judge/games/{code}/formats	# Formatos do jogo
GET	/runtime/judge/games/{code}/cards/search?q=	# Busca de cards
POST	/runtime/judge/games/{code}/decklist/parse	# Parse decklist
POST	/runtime/judge/games/{code}/decklist/validate	# Validar decklist
GET	/runtime/judge/games/{code}/banlist	# Banlist
POST	/runtime/judge/admin/sync-cards/{game}	# Sync manual de cards

4.3 Player Ecosystem

GET	/runtime/judge/players/{handle}	# Perfil público
GET	/runtime/judge/players/{handle}/stats	# Estatísticas
GET	/runtime/judge/players/{handle}/history	# Histórico de torneios
GET	/runtime/judge/players/{handle}/achievements	# Conquistas
GET	/runtime/judge/players/{handle}/rankings	# Rankings
GET	/runtime/judge/players/{handle}/decklists	# Decklists públicas
PUT	/runtime/judge/players/me	# Atualizar perfil
PUT	/runtime/judge/players/me/privacy	# Configurar privacidade
GET	/runtime/judge/leaderboards/{game}/{format}	# Leaderboards
GET	/runtime/judge/search/tournaments	# Busca de torneios

4.4 Social

POST	/runtime/judge/social/friends/request	# Enviar solicitação
POST	/runtime/judge/social/friends/accept	# Aceitar
POST	/runtime/judge/social/friends/reject	# Rejeitar
DELETE	/runtime/judge/social/friends/{id}	# Remover amigo
GET	/runtime/judge/social/friends	# Lista de amigos
WS	/ws/chat	# Chat em tempo real
GET	/runtime/judge/social/messages	# Histórico de mensagens
POST	/runtime/judge/social/communities	# Criar comunidade
GET	/runtime/judge/social/communities	# Listar
POST	/runtime/judge/social/communities/{id}/join	# Entrar

4.5 Marketplace & Stores

POST	/runtime/judge/stores	# Criar loja
GET	/runtime/judge/stores	# Listar lojas
GET	/runtime/judge/stores/{slug}	# Perfil da loja
PUT	/runtime/judge/stores/{id}	# Atualizar
POST	/runtime/judge/stores/{id}/verify	# Solicitar verificação
POST	/runtime/judge/stores/{id}/subscribe	# Assinar plano

GET	/runtime/judge/stores/{id}/tournaments	# Torneios da loja
GET	/runtime/judge/stores/{id}/reviews	# Avaliações
POST	/runtime/judge/marketplace/decklists	# Publicar decklist
GET	/runtime/judge/marketplace/decklists	# Listar
GET	/runtime/judge/marketplace/decklists/{id}	# Detalhes
POST	/runtime/judge/marketplace/decklists/{id}/buy	# Comprar
POST	/runtime/judge/reviews	# Criar review
GET	/runtime/judge/reviews/{targetType}/{targetId}	# Listar reviews

4.6 Payments

POST	/runtime/judge/payments/intent	# Criar intenção de pagamento
POST	/runtime/judge/payments/confirm	# Confirmar (webhook)
POST	/runtime/judge/payments/refund	# Reembolso
GET	/runtime/judge/payments/{id}/status	# Status
GET	/runtime/judge/organizers/me/payouts	# Histórico de payouts
GET	/runtime/judge/organizers/me/revenue	# Dashboard de receita

4.7 Public API (Developer Portal)

GET	/public/v1/tournaments	# Listar torneios públicos
GET	/public/v1/tournaments/{id}	# Detalhes
GET	/public/v1/tournaments/{id}/standings	# Standings
GET	/public/v1/tournaments/{id}/pairings	# Pairings
GET	/public/v1/players/{handle}	# Perfil público
GET	/public/v1/leaderboards/{game}/{format}	# Leaderboards
GET	/public/v1/cards/{game}/search	# Busca de cards
GET	/public/v1/decklists/marketplace	# Decklists à venda
POST	/public/v1/webhooks	# Registrar webhook
DELETE	/public/v1/webhooks/{id}	# Remover webhook

4.8 Overlay (OBS)

GET	/overlay/{tournamentId}/standings	# Standings em tempo real
GET	/overlay/{tournamentId}/pairings	# Pairings atuais
GET	/overlay/{tournamentId}/timer	# Timer
GET	/overlay/{tournamentId}/bracket	# Bracket
WS	/overlay/ws/{tournamentId}	# WebSocket overlay

4.9 Admin

GET	/runtime/judge/admin/stats	# Estatísticas da plataforma
GET	/runtime/judge/admin/users	# Listar usuários
POST	/runtime/judge/admin/users/{id}/ban	# Banir usuário
POST	/runtime/judge/admin/tournaments/{id}/moderate	# Moderar torneio
POST	/runtime/judge/admin/sync-cards	# Forçar sync

POST	/runtime/judge/admin/broadcast	# Notificação global
GET	/runtime/judge/admin/payments	# Transações
GET	/runtime/judge/admin/logs	# Logs de auditoria

5. FRONTEND ROUTES

/	# Landing page com TournamentShowcase
/search	# Descoberta de torneios
/tournament/create	# Wizard de criação de torneio
/tournament/[id]	# Dashboard do torneio (organizador)
/tournament/[id]/play	# Vista do jogador (pairings + timer)
/player/[handle]	# Perfil público do jogador
/player/me	# Perfil próprio (edição)
/stores	# Listagem de lojas
/stores/[slug]	# Perfil da loja
/stores/create	# Criar loja
/marketplace	# Decklists à venda
/marketplace/[id]	# Detalhe do decklist
/leagues	# Listagem de ligas
/leagues/create	# Criar liga (wizard 12 semanas)
/leagues/[id]	# Visualização da liga
/social	# Comunidades + chat drawer
/social/friends	# Lista de amigos
/social/communities	# Comunidades
/developer	# Developer portal (API keys, webhooks)
/overlay/[tournamentId]	# Configuração de overlay OBS
/dashboard/organizer	# Analytics do organizador
/admin	# Painel administrativo
/pos	# Interface POS (tablet/desktop loja)
/judge	# Painel do juiz (Fase 6)

6. COMPONENTES FRONTEND PRINCIPAIS

6.1 Tournament

src/components/tournament/	
└─ GameSelector.tsx	# Seletor de jogo com ícones
└─ TournamentCreateForm.tsx	# Formulário de criação
└─ RoundManager.tsx	# Controle de rodadas (organizador)
└─ TimerDisplay.tsx	# Display de timer
└─ StandingsTable.tsx	# Tabela de standings

```
|— PairingsView.tsx      # Visualização de pairings
|— ResultReporter.tsx    # Reporte de resultados B03
|— BracketTree.tsx       # Árvore de eliminação
└— TournamentShowcase.tsx # Showcase na landing
```

6.2 Player

```
src/components/player/
|— PlayerProfile.tsx      # Perfil público
|— StatsCard.tsx         # Card de estatísticas
|— AchievementBadge.tsx   # Badge de conquista
|— TournamentHistory.tsx  # Histórico de torneios
|— RankingDisplay.tsx     # Display de ranking
└— DecklistCard.tsx       # Card de decklist
```

6.3 Marketplace

```
src/components/marketplace/
|— DecklistCard.tsx      # Card de decklist à venda
|— DecklistDetail.tsx    # Detalhe do decklist
|— PurchaseModal.tsx     # Modal de compra
└— ReviewCard.tsx        # Card de review
```

6.4 Streaming

```
src/components/streaming/
|— OverlayConfig.tsx     # Configuração OBS
└— OverlayPreview.tsx    # Preview de overlay
```

6.5 Social

```
src/components/social/
|— FriendButton.tsx      # Botão adicionar/remover amigo
|— ChatDrawer.tsx        # Drawer de chat
|— CommunityCard.tsx     # Card de comunidade
└— MessageBubble.tsx     # Bolha de mensagem
```

6.6 Notifications

```
src/components/notifications/
└— PermissionPrompt.tsx  # Solicitação de permissão push
```

6.7 Mobile

```
src/components/mobile/
|— MobileLayout.tsx      # Layout mobile com bottom nav
└— MobilePairingCard.tsx # Card de pairing otimizado mobile
```

6.8 Onboarding

```
src/components/onboarding/  
└─ OnboardingFlow.tsx          # Fluxo de 5 passos
```

7. HOOKS FRONTEND

```
src/hooks/  
├─ useTournamentGames.ts      # Jogos e formatos disponíveis  
├─ useTournamentFlow.ts       # Fluxo completo de torneio  
├─ useRoundTimer.ts           # Timer via WebSocket  
├─ usePairings.ts             # Pairings ativos  
├─ usePlayerProfile.ts        # Perfil de jogador  
├─ useRankings.ts             # Rankings e leaderboards  
├─ useNotifications.ts        # Notificações  
├─ usePayments.ts             # Pagamentos  
├─ useOrganizerAnalytics.ts    # Analytics do organizador  
├─ useTournamentSearch.ts     # Busca de torneios  
├─ useFriendship.ts           # Estado de amizade  
├─ useChat.ts                 # Chat em tempo real  
├─ useLeague.ts               # Dados de liga  
└─ useAdmin.ts                # Dados administrativos
```

8. BACKEND MODULES

8.1 Tournament Engine

```
# services/api/app/tournament/  
engine/  
┌─ swiss.py          # Algoritmo Swiss  
│ # - Filtra checked-in participants  
│ # - Ordena por match_points DESC, tiebreakers DESC  
│ # - Bye para menor pontuação  
│ # - Pareamento por score groups  
│ # - Anti-rematch (forced_rematch flag se necessário)  
│ # - Table number sequencial  
└─ tiebreakers.py    # Cálculo de tiebreakers  
  # - OMW% (Opponent Match Win %)  
  # - GW% (Game Win %)  
  # - OGW% (Opponent Game Win %)  
  # - Aplicação de resultados em standings  
┌─ bracket.py        # Bracket eliminatório  
│ # - Top N seeding: (1vsN), (2vsN-1), etc.  
│ # - Avanço automático de vencedores  
│ # - Suporte a terceiro lugar
```

```

timer.py                # Timer Redis + Pub/Sub
# - start(), extend(), remaining_seconds()
# - Broadcast via WebSocket/SSE
# - Alertas: 5min, 1min, ended

flow.py                 # Orquestração do ciclo de vida
# - start_check_in() → start() → generate_round() → end_round()
# → calculate_standings() → advance_top_cut() → finalize()

store.py                # Persistência Postgres

```

8.2 Game Adapters

```

# services/api/app/tcg_adapters/
base.py, types.py, registry.py    # Interface GameAdapter + registry

pokemon.py                # Regras Pokémon TCG
# - 60 cartas exatas
# - 4 cópias (exceto Basic Energy ilimitada)
# - 1 Pokémon Básico mínimo
# - Rotation Standard/Expanded

lorcana.py               # Regras Disney Lorcana
# - 60 cartas exatas
# - 4 cópias por nome
# - Chapter rotation

mtg.py                   # Regras Magic: The Gathering
# - 60+ maindeck (40 Limited, 100 Commander)
# - 4 cópias (1 Commander)
# - Sideboard 0-15
# - Color identity (Commander)

swu.py                   # Regras Star Wars Unlimited
# - 60 cartas
# - 4 cópias
# - Aspect rules

parsers.py               # Parse PTCGO / .dec / texto / JSON
validation.py            # Validação contra card_catalog + banlist

```

8.3 Player Ecosystem

```

# services/api/app/players/
store.py                 # CRUD perfil, handle único
rankings.py              # Cálculo de pontos, tiers, decay
achievements.py          # Desbloqueio de conquistas

```

8.4 Social

```

# services/api/app/social/

```

```
friendships.py      # CRUD amizades
messages.py         # Chat WebSocket
communities.py      # CRUD comunidades
```

8.5 Marketplace

```
# services/api/app/marketplace/
decklists.py        # CRUD decklists
sales.py            # Fluxo de compra (70% vendedor / 30% plataforma)
reviews.py          # Reviews de torneios/lojas/decklists
```

8.6 Payments

```
# services/api/app/payments/
tournament_entry.py # Stripe intent, confirmação, reembolso
# - Reembolso 100% > 24h antes
# - Reembolso 50% < 24h
# - No-show = sem reembolso
payouts.py          # Payouts para organizadores (D+2)
```

8.7 Notifications

```
# services/api/app/notifications/
service.py          # Orquestração
push.py             # Web Push VAPID
email.py            # SendGrid/SES (stub)
sms.py              # Twilio (stub)
```

8.8 Public API

```
# services/api/app/public_api/
auth.py             # API Key auth
rate_limit.py       # Rate limiting Redis
webhooks.py         # Webhook dispatch com retry
```

8.9 Overlay

```
# services/api/app/overlay/
websocket.py        # WS para OBS
# Templates HTML/CSS em overlay/templates/
```

8.10 Admin

```
# services/api/app/admin/
dashboard.py        # Stats da plataforma
moderation.py       # Moderação de torneios/usuários
audit.py            # Logs de auditoria
```

8.11 Jobs

```
# services/api/app/jobs/
ranking_decay.py      # Decay diário 5% (04:00 UTC)
card_sync.py          # Sync Scryfall/TCGdex/Lorcana
```

9. MIGRATIONS (Ordem de Aplicação)

```
supabase/migrations/
├─ 20260605100000_multi_tcg_tournament.sql      # Fase 1: games, formats, card_catalog, tournaments
├─ 20260605120000_tournament_lifecycle.sql      # Fase 2: rounds, pairings, brackets, participants
├─ 20260605140000_player_ecosystem.sql          # Fase 3: profiles, stats, achievements, rankings,
↪ payments
├─ 20260605160000_production_social_rls.sql     # Fase 4: RLS, social, mobile, push, admin
├─ 20260605180000_marketplace_streaming_api.sql # Fase 5: stores, marketplace, reviews, api_keys,
↪ webhooks, sponsorships
```

Comando para aplicar:

```
cd S:\tcg-judge\supabase
supabase link --project-ref <seu-ref>
supabase db push
```

10. REGRAS DE NEGÓCIO CRÍTICAS

10.1 Torneios

#	Regra
R1	Jogador só pode estar ativo em um torneio por vez
R2	Decklist não pode ser alterada após check-in/start
R3	Pairings só gerados se checked-in ≥ 2
R4	Bye = 3 pts, não afeta tiebreakers de oponentes
R5	Organizador pode DQ jogador a qualquer momento
R6	Resultado disputado fica pending até intervenção do organizador
R7	Torneios públicos aparecem na landing; privados só por link

10.2 Pagamentos

#	Regra
R8	Inscrição só confirmada após pagamento

Table 2 – continued

#	Regra
R9	Cancelamento > 24h = reembolso 100%
R10	Cancelamento < 24h = reembolso 50%
R11	Organizador cancela = reembolso 100%
R12	No-show = sem reembolso
R13	Payout organizador em D+2

10.3 Marketplace

#	Regra
R14	Vendedor recebe 70%, plataforma 30%
R15	Decklist entregue após pagamento confirmado
R16	Review só pode ser deixada por comprador

10.4 Rankings

#	Regra
R17	Decay 5% mensal por inatividade
R18	Pontos baseados em posição × tamanho do torneio
R19	Bônus 10% para torneios oficiais

11. PLANO DE ASSINATURAS

Organizadores (Lojas)

Plano	Mensal	Torneios	Comissão	Features
Free	R\$ 0	5/mês	10%	Básico
Pro	R\$ 49	Ilimitado	7%	+Analytics, branding, suporte
Enterprise	R\$ 199	Ilimitado	5%	+API, white-label, conta dedicada

Developers (API)

Plano	Mensal	Requests	Features
Free	R\$ 0	10k/mês	Básico
Pro	R\$ 29	50k/mês	+Rate limit, webhooks
Enterprise	Custom	Ilimitado	+SLA, suporte

12. TESTES

Suite	Total	Status
pytest (tournament + tcg_adapters)	51	✓
pytest (players + social + admin + jobs)	22	✓
pytest (stores + marketplace + public_api + sponsorships)	24	✓
pytest TOTAL	97	✓
vitest (total frontend)	109	✓
TOTAL	206	✓

Estrutura de Testes

```
tests/
├─ tournament/      # Engine Swiss, tiebreakers, bracket, timer, flow
├─ tcg_adapters/    # Parsers, validation, sync
├─ players/         # Profile, rankings, achievements
├─ social/          # Friendships, messages, communities
├─ marketplace/     # Decklists, sales, reviews
├─ payments/        # Stripe integration
├─ public_api/      # Rate limiting, auth
├─ admin/           # Dashboard, moderation
└─ jobs/            # Ranking decay, card sync
```

13. PRÓXIMOS PASSOS (FASE 6)

Item	Prioridade	Complexidade
Mobile app (React Native)	Alta	Alta
IA recomendação de decklists	Alta	Média
Análise de metagame	Média	Média
Integração POS (Square/Stripe Terminal)	Média	Média
Sistema de juiz digital	Alta	Média
i18n (EN, ES, JA)	Média	Baixa
Fair play / infrações	Média	Média
Backup e DR	Alta	Baixa

14. DECISÕES ARQUITETURAIS

1. **Multi-TCG via GameAdapter:** Cada jogo implementa interface comum, permitindo extensão futura

2. **Swiss Engine custom:** Greedy algorithm suficiente para TCG (até 256 jogadores). Blossom algorithm se necessário no futuro
 3. **Redis para timer:** Persistência entre reinícios do servidor, fallback para memória
 4. **WebSocket + SSE:** WS para operações bidirecionais, SSE para broadcast unidirecional
 5. **Supabase Auth + RLS:** Autenticação e autorização no nível do banco
 6. **BFF Pattern:** Next.js API Routes como proxy para backend FastAPI
 7. **PWA + Mobile App:** PWA para web, React Native para nativo (compartilha lógica)
 8. **Stripe para pagamentos:** Suporte a cartão, PIX (BR), PayPal
 9. **VAPID para Web Push:** Padrão aberto, funciona em todos os navegadores modernos
-

15. CONVENÇÕES DE CÓDIGO

Python (Backend)

- PEP 8, type hints obrigatórios
- Async/await para I/O
- SQLAlchemy 2.0 ou raw SQL parametrizado
- pytest para testes, coverage > 80%

TypeScript (Frontend)

- Strict mode ativado
- Componentes funcionais com hooks
- Tailwind CSS para estilização
- Vitest + React Testing Library

Database

- Snake_case para tabelas e colunas
- UUID para PKs
- JSONB para dados flexíveis
- Índices para campos de busca frequentes
- RLS em todas as tabelas com dados sensíveis

Git

- Conventional commits: `feat:`, `fix:`, `refactor:`, `test:`, `docs:`
 - Branch naming: `feature/`, `bugfix/`, `hotfix/`
 - PRs com descrição e testes
-

16. AMBIENTES

Ambiente	URL	Propósito
Local	http://localhost:3000	Desenvolvimento
Staging	https://staging.tcg-judge.com	QA/Preview
Production	https://tcg-judge.com	Produção

Variáveis de Ambiente

- `.env.local` — desenvolvimento
 - `.env.staging` — staging
 - `.env.production` — produção (nunca commitar)
 - `.env.production.example` — template (commitar)
-

17. COMANDOS ÚTEIS

```
# Backend
cd services/api
pip install -r requirements.txt
uvicorn app.main:app --reload

# Frontend
cd frontend/runtime_console_v3
npm install
npm run dev

# Tests
pytest                # Backend
npm run test           # Frontend

# Database
cd supabase
supabase db push
supabase db reset

# Worker
python services/api/worker_main.py

# Docker
docker-compose -f docker/docker-compose.prod.yml up
```

18. CONTATOS E RECURSOS

- **Repositório:** `S:\tcg-judge`
 - **Documentação:** `docs/` (`README.md`, `ARCHITECTURE.md`, `API.md`, `DEPLOYMENT.md`, `API_PUBLIC.md`)
 - **Design System:** Tailwind CSS + componentes em `src/components/ui/`
 - **Ícones:** Lucide React
 - **Fonte:** Inter (Google Fonts)
-

Este arquivo de contexto deve ser mantido atualizado conforme o projeto evolui. Última atualização: 2026-06-06